

JAVA INTERVIEW QUESTIONS

1. Explain JDK, JRE and JVM?

JDK	JRE	JVM
It stands for Java Development Kit.	It stands for Java Runtime Environment.	It stands for Java Virtual Machine.
It is the tool necessary to compile, document and package Java programs.	JRE refers to a runtime environment in which Java bytecode can be executed.	It is an abstract machine. It is a specification that provides a run-time environment in which Java bytecode can be executed.
It contains JRE + development tools.	It's an implementation of the JVM which physically exists.	JVM follows three notations: Specification, Implementation, and Runtime Instance.

2. Explain public static void main(String args[]) in Java.

- **public:**
 - Public is an access modifier, which is used to specify who can access this method.
 - Public means that this Method will be accessible by any Class.
- **static:**
 - It is a keyword in java which identifies it is class-based.
 - main() is made static in Java so that it can be accessed without creating the instance of a Class.
 - In case, main is not made static then the compiler will throw an error as **main()** is called by the JVM before any objects are

NEWGEN CORPORATE TRAINING CENTER

made and only static methods can be directly invoked via the class.

- **void:**
 - It is the return type of the method.
 - Void defines the method which will not return any value.
- **main:**
 - It is the name of the method which is searched by JVM as a starting point for an application with a particular signature only.
 - It is the method where the main execution occurs.
- **String args[]:** It is the parameter passed to the main method.

QUE :What if we not written a Static in above question?

ANS :It will throw an Error

3. Why Java is not 100% Object-oriented?

- Because of Primitive Data Type
- e.g. boolean, byte, char, int, float, double, long, short which are not objects.
- Without creating Object we can run a program in JAVA , so 100 % Not Object oriented.

4. What are wrapper classes in Java?

the **wrapper class in Java** provides the mechanism *to convert primitive into object and object into primitive.*

the eight classes of the *java.lang* package are known as wrapper classes in Java. The list of eight wrapper classes are given below:

Primitive Type	Wrapper class
boolean	Boolean
char	Character

NEWGEN CORPORATE TRAINING CENTER

byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

5. What are constructors in Java?

- Method name and class name are same
- A Constructor not having any Return type.
- A Java constructor cannot be **abstract, static, final, and synchronized**
- It gets automatically Execute when object is Initialized.
- It is a special type of method which is used to initialize the object.

6. What is singleton class in Java

- Singleton class is a class whose only one instance can be created at any given time, in one JVM.
- A class can be made singleton by making its constructor private.

7. What is the difference between a local variable and an instance variable?

- **Local Variable**
 - **local variable** is typically used inside a method, constructor, or a **block** and has only local scope.
 - Thus, this variable can be used only within the scope of a block.

NEWGEN CORPORATE TRAINING CENTER

- The best benefit of having a local variable is that other methods in the class won't be even aware of that variable.

Instance Variable :

- It is a variable which is bounded to its object itself.
- These variables are declared within a **class**, but outside a method.
- Every object of that class will create it's own copy of the variable while using it.

8. Differentiate between the constructors and methods in Java?

Methods	Constructors
1. Used to represent the behavior of an object	1. Used to initialize the state of an object
2. Must have a return type	2. Do not have any return type
3. Needs to be invoked explicitly	3. Is invoked implicitly

NEWGEN CORPORATE TRAINING CENTER

4. No default method is provided by the compiler

4. A default constructor is provided by the compiler if the class has none

5. Method name may or may not be same as class name

5. Constructor name must always be the same as the class name

9. What is final keyword in Java?

Final is a keyword Applicable to Method, Variable, class

- **final variable**

- When the final keyword is used with a variable then its value can't be changed once assigned.
- If you tried to change it, it will throw an error.

- **final method**

- When a method is declared as a final then it can't be overridden.

- **final class**

- We can't Inherited a class.

10. What is the difference between this() and super() in Java?

this()	super()
1. this() represents the current instance of a class	1. super() represents the current instance of a parent/base class
2. Used to call the default constructor of the same class	2. Used to call the default constructor of the parent/base class
3. Used to access methods of the current class	3. Used to access methods of the base class

NEWGEN CORPORATE TRAINING CENTER

4. Used for pointing the current class instance 4. Used for pointing the superclass instance

11. Differentiate between static and non-static methods in Java.

Static Method	Non-Static Method
1. The static keyword must be used before the method name	1. No need to use the static keyword before the method name
2. It is called using the class (className.methodName)	2. It can be called like any general method
3. They can't access any non-static instance variables or methods	3. It can access any static method and any static variable without creating an instance of the class

12. Difference between String, StringBuilder, and StringBuffer.

Factor	String	StringBuilder	StringBuffer
Storage Area	Constant String Pool	Heap Area	Heap Area
Mutability	Immutable	Mutable	Mutable
Thread Safety	Yes	No	Yes
Performance	Fast	More efficient	Less efficient

13. Why Java Strings are immutable in nature?

- Immutable simply means unmodifiable or unchangeable

NEWGEN CORPORATE TRAINING CENTER

- Once string object is created its data or state can't be changed but a new string object is created.
- For Security Purpose String is made Immuatable.

14. What is the difference between an array and an array list?

Array	ArrayList
Cannot contain values of different data types	Can contain values of different data types.
Size must be defined at the time of declaration	Size can be dynamically changed
Need to specify the index in order to add data	No need to specify the index
Arrays are not type parameterized	Arraylists are type
Arrays can contain primitive data types as well as objects	Arraylists can contain only objects, no primitive data types are allowed
...	...

15. What is a Map in Java?

- Map is an interface of Util package which maps unique keys to values.
- The Map interface is not a Collection interface

16. What is collection class in Java?

- collection is a framework used for storing and manipulating a group of objects.

NEWGEN CORPORATE TRAINING CENTER

- Using Collections you can perform various tasks like searching, sorting, insertion, manipulation, deletion, etc.

17. What is Polymorphism?

- Polymorphism is briefly described as “one interface, many implementations”.
- There are two types of Polymorphism .

1. Compile time polymorphism / method overloading

2. Run time polymorphism / Method Over Ridding

18. What is Method Over Loading / Compile time polymorphism ?

- Method Overloading means Same Method Name but different In arguments.
- It is in Method should be declared in Same Class.
- It is used in order to add more to the behavior of methods.

19. What is method Over Ridding / Run time Polymorphism ?

- Same Method Name and Same Arguments.
- inheritance always required.
- it is used in order to change the behavior of exist methods.

20. Difference between Method Over Load and Method Over Ridding

S.NO	Method Overloading	Method Overriding
1.	In the method overloading, methods or functions must have the same name and	Whereas in the method overriding, methods or functions must have the same name and

NEWGEN CORPORATE TRAINING CENTER

S.NO	Method Overloading	Method Overriding
	different signatures.	same signatures.
2.	Method overloading is a example of compile time polymorphism.	Whereas method overriding is a example of run time polymorphism.
3.	In the method overloading, inheritance may or may not be required.	Whereas in method overriding, inheritance always required.
4.	Method overloading is performed between methods within the class.	Whereas method overriding is done between parent class and child class methods.
5.	It is used in order to add more to the behavior of methods.	Whereas it is used in order to change the behavior of exist methods.
6.	In method overloading, there is no need of more than one class.	Whereas in method overriding, there is need of at least of two classes.

21. Can we overload methods on return type?

Ans :No

- Ambiguity Error comes, whether their return-type is different or not.so We cant OverLoad on Same Return type.

22. Can we overload static methods?

- The answer is 'Yes'.
- We can have two or more static methods with same name, but differences in input parameters.

```
Public Static void x(int a){  
  
}  
  
Public static void x(String a){  
  
}
```

23. Can we overload main() in Java?

Yes. we can overload main() in Java

24. Can we Over ride Private method.

- No. Private methods can not be overridden
- For OverRidden method we need inheritance , but if we make Parent method as Private , then We can't Access those method.

25. Can We Over Ride Static methods

- No.
- Static methods can not be override
- Static Is Kayword , it is used at class level ,not object level.
- If we want to over ride any method we need to create object first then we override that method , which is not possible with Static Keyword.

26. Can we Over Ride Final Method ?

NEWGEN CORPORATE TRAINING CENTER

- Final methods can not be overridden
- Methods are declared final in java to prevent subclasses from Overriding them and changing their behavior, the reason this works is discussed at the end of this article.
- The advantage of the final keyword is that it stops developers from intentionally or unintentionally changing the behavior of methods that should not be changed for security or other reasons.

27. Can We Over ride Constructor ?

No

28. Can we overload methods on return type?

We **cannot** overload by return type

29. Can we overload static methods?

The answer is '**Yes**'. We can have two ore more static methods with same name, but differences in input parameters.

30. Can we overload main() in Java?

Like other static methods, we can overload main() in Java

31. What is abstraction in Java?

- **abstraction** is the process of hiding certain details and showing only essential information to the user.
- There are two ways to achieve abstraction in java
 - Abstract class (0 to 100%)
 - Interface (100%)

NEWGEN CORPORATE TRAINING CENTER

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

E.g ATM Machine

32. Advantages of Abstraction

- It reduces the complexity of viewing the things.
- Avoids code duplication and increases reusability.
- Helps to increase security of an application or program as only important details are provided to the user.

33. What is Interface ?

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.
- we can have **default and static methods** in an interface
- You cannot instantiate an interface.
- An interface does not contain any constructors.
- All of the methods in an interface are abstract.
- An interface cannot contain instance fields. The only fields that can appear in an interface must be declared both static and final.
- An interface is not extended by a class; it is implemented by a class.

NEWGEN CORPORATE TRAINING CENTER

34. What is the difference between abstract classes and interfaces?

Abstract Class	Interfaces
An abstract class can provide complete, default code and/or just the details that have to be overridden	An interface cannot provide any code at all, just the signature
In the case of an abstract class, a class may extend only one abstract class	A Class may implement several interfaces
An abstract class can have non-abstract methods	All methods of an Interface are abstract
An abstract class can have instance variables	An Interface cannot have instance variables
An abstract class can have any visibility: public, private, protected	An Interface visibility must be public (or) none
If we add a new method to an abstract class then we have the option of providing default implementation and therefore all the existing code might work properly	If we add a new method to an Interface then we have to track down all the implementations of the interface and define implementation for the new method
An abstract class can contain constructors	An Interface cannot contain constructors
Abstract classes are fast	Interfaces are slow as it requires extra indirection to find the corresponding method in the actual class

35. What is inheritance in Java?

NEWGEN CORPORATE TRAINING CENTER

- Inheritance in Java is the concept where the properties of one class can be inherited by the other
- A class which inherits the properties is known as Child Class whereas a class whose properties are inherited is known as Parent class.
- **Types of Inheritance**
 - **Single Inheritance:**
 - **Multilevel Inheritance:**
 - **Hierarchical Inheritance:**
 - **Hybrid Inheritance:**

36. Why is Inheritance used in Java?

- Inheritance provides code reusability.
- Runtime polymorphism cannot be achieved without using inheritance.
- We can simulate the inheritance of classes with the real-time objects which makes OOPs more realistic.
- Inheritance provides data hiding. The base class can hide some data from the derived class by making it private.
- Method overriding cannot be achieved without inheritance. By method overriding, we can give a specific implementation of some basic method contained by the base class.

37. What is encapsulation in Java?

- **Encapsulation** is a mechanism where you bind your data(variables) and code(methods) together as a single unit.
- Here, the data is hidden from the outer world and can be accessed only via current class methods.
- We can achieve encapsulation in Java by:
 - Declaring the variables of a class as private.

NEWGEN CORPORATE TRAINING CENTER

- Providing public setter and getter methods to modify and view the values of the variables.

38. What are the various access specifiers in Java?

- **Public** The classes, methods, or variables which are defined as public, can be accessed by any class or method.
- **Protected** Protected can be accessed by the class of the same package, or by the sub-class of this class, or within the same class.
- **Default** Default are accessible within the package only. By default, all the classes, methods, and variables are of default scope.
- **Private** The private class, methods, or variables defined as private can be accessed within the class only

39. What is the purpose of static Keyword..

static keyword is used with variables, methods, blocks and nested classes.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- A static method can be invoked without the need for creating an instance of a class.
- A static method belongs to the class rather than the object of a class.
- The static method can not use non static data member or call non-static method directly.
- this and super cannot be used in static context.

40. What is Java static block

- It Is used to initialize the static data member.

NEWGEN CORPORATE TRAINING CENTER

- It is executed before the main method at the time of class loading.

41. Can we execute a program without main() method?

- it is not possible to execute a Java class without the main method.

42. What is This Keyword in JAVA

- this can be used to refer current class instance variable.
- this can be used to invoke current class method (implicitly)
- this() can be used to invoke current class constructor.
- this can be passed as an argument in the method call.
- this can be passed as argument in the constructor call.
- this can be used to return the current class instance from the method.

43. What is Super keyword in java

- Super can be used to refer immediate parent class instance variable.
- super can be used to invoke immediate parent class method.
- super() can be used to invoke immediate parent class constructor.

44. What are the advantages of Packages in Java?

- Packages avoid the name clashes.
- The Package provides easier access control.
- We can also have the hidden classes that are not visible outside and used by the package.
- It is easier to locate the related classes.

45. Is constructor inherited?

NEWGEN CORPORATE TRAINING CENTER

- No, The constructor is not inherited.

46. Can you make a constructor final?

- No, the constructor can't be final.

47. Can we overload the constructors?

- Yes, the constructors can be overloaded by changing the number of arguments accepted by the constructor or by changing the data type of the parameters

48. What is the difference between static (class) method and instance method?

static or class method	instance method
1)A method that is declared as static is known as the static method.	A method that is not declared as static is known as the instance method.
2)We don't need to create the objects to call the static methods.	The object is required to call the instance methods.
3)Non-static (instance) members cannot be accessed in the static context (static method, static block, and static nested class) directly.	Static and non-static variables both can be accessed in instance methods.
4)For example: <code>public static int cube(int n){ return n*n*n;}</code>	For example: <code>public void msg(){...}.</code>

48. Can we make constructors static?

NEWGEN CORPORATE TRAINING CENTER

- As we know that the static context (method, block, or variable) belongs to the class, not the object.
 - Since Constructors are invoked only when the object is created, there is no sense to make the constructors static.
 - However, if you try to do so, the compiler will show the compiler error.
-

49. Can we make the abstract methods static in Java?

- if we make the abstract methods static, It will become the part of the class, and we can directly call it which is unnecessary.
- Calling an undefined method is completely useless therefore it is not allowed.

50. Can we declare the static variables and methods in an abstract class?

- Yes, we can declare static variables and methods in an abstract method.
- As we know that there is no requirement to make the object to access the static context, therefore, we can access the static context declared inside the abstract class by using the name of the abstract class.

51. Which class is the superclass for all the classes?

- The object class is the superclass of all other classes in Java.

52. What are the differences between this and super keyword?

- The **super** keyword always points to the parent class contexts whereas **this** keyword always points to the current class context.
- The **super** keyword is primarily used for initializing the base class variables within the derived class constructor whereas **this** keyword

NEWGEN CORPORATE TRAINING CENTER

primarily used to differentiate between local and instance variables when passed in the class constructor.

- The **super** and **this** must be the first statement inside constructor otherwise the compiler will throw an error.

53. What is the difference between abstraction and encapsulation?

- Abstraction hides the implementation details whereas encapsulation wraps code and data into a single unit.

54. Can you use abstract and final both with a method?

- No, because we need to override the abstract method to provide its implementation, whereas we can't override the final method.

55. Can you declare an interface method static?

- No,
- because methods of an interface are abstract by default, and
- we can not use static and abstract together.

56. Can the Interface be final?

- No,
- because an interface needs to be implemented by the other class and
- if it is final, it can't be implemented by any class.

57. Can we define private and protected modifiers for the members in interfaces?

- No, they are implicitly public.

58. How many types of exception can occur in a Java program?

NEWGEN CORPORATE TRAINING CENTER

- There are mainly two types of exceptions:
 - checked and
 - unchecked.

Here, an error is considered as the unchecked exception.
- **Checked Exception:**
 - Checked exceptions are the one which are checked at compile-time.
 - For example, SQLException, ClassNotFoundException, etc.
- **Unchecked Exception:**
 - Unchecked exceptions are the one which are handled at runtime because they can not be checked at compile-time.
 - For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc.
- **Error:** Error cause the program to exit since they are not recoverable. For Example, OutOfMemoryError, AssertionError, etc.

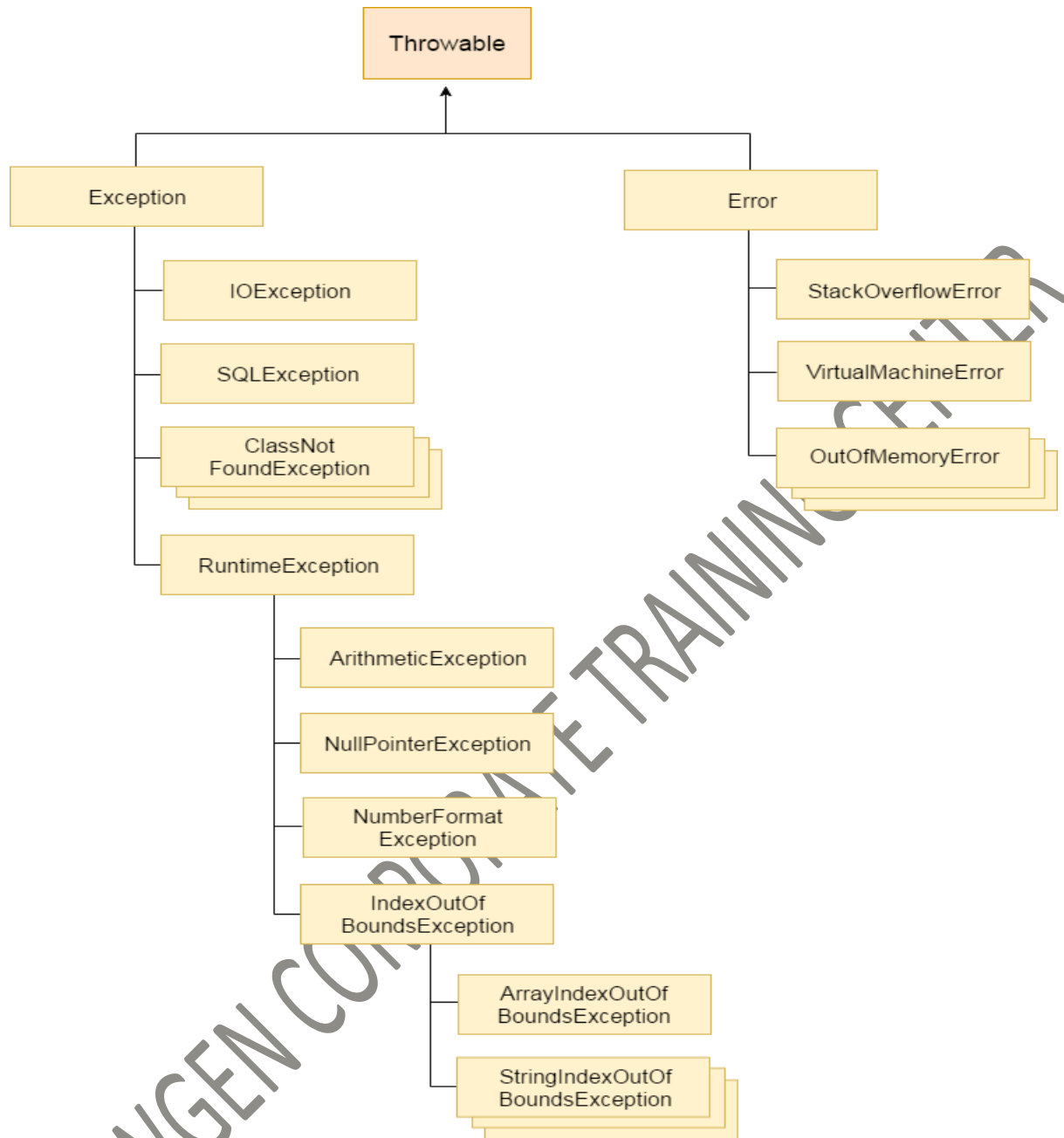
59. What is Exception Handling?

- Exception Handling is a mechanism that is used to handle runtime errors.
- It is used primarily to handle checked exceptions.
- Exception handling maintains the normal flow of the program.
- We can Achieve Exception Handling using **Try Catch Block**.

60. Explain the hierarchy of Java Exception classes?

Throwable class is the root(**Parent**) class of Java Exception hierarchy

NEWGEN CORPORATE TRAINING CENTER



61. What is the difference between Checked Exception and Unchecked Exception?

Difference Between Checked And Unchecked Exceptions

Exceptions that are checked and handled at compile time are checked exceptions.	Exceptions that are not checked and handled at compile time are unchecked exceptions.
They are direct subclasses of Exception but do not inherit from RuntimeException.	They are a direct subclass of RuntimeException class.
The program gives a compilation error if a method throws a checked exception and the compiler is not able to handle the exception on its own.	The program compiles fine because the exceptions escape the notice of compiler. Exceptions occur due to errors in programming logic.
A Checked Exception occurs when the chances of failure are too high.	Unchecked Exception occurs mostly due to programming mistakes.
Common checked exceptions include IOException, DataAccessException, IllegalAccessException, InterruptedException, etc.	Common unchecked exceptions include ArithmeticException, InvalidClassException, NullPointerException, etc.

62. What is the base class for Error and Exception?



- The Throwable class is the base class for Error and Exception.

63. What is finally block?

- The "finally" block is used to execute the important code of the program.
- It is executed whether an exception is handled or not.

64. What is the difference between throw and throws?

throw keyword	throws keyword
1) The throw keyword is used to throw an exception explicitly.	The throws keyword is used to declare an exception.
2) The checked exceptions cannot be propagated with throw only.	The checked exception can be propagated with throws
3) The throw keyword is followed by an instance.	The throws keyword is followed by class.
4) The throw keyword is used within the method.	The throws keyword is used with the method signature.
5) You cannot throw multiple exceptions.	You can declare multiple exceptions, e.g., public void method()throws IOException, SQLException.

65. What is the difference between final, finally and finalize?

No.	final	finally	finalize
1)	Final is used to apply restrictions on class, method, and variable. The final class can't be inherited, final method can't be overridden, and final variable value can't be changed.	Finally is used to place important code, it will be executed whether an exception is handled or not.	Finalize is used to perform clean up processing just before an object is garbage collected.
2)	Final is a keyword.	Finally is a block.	Finalize is a method.

66. What is the difference between Iterator and ListIterator?

- Iterator traverses the elements in the forward direction only whereas
- ListIterator traverses the elements into forward and backward direction.

No.	Iterator	ListIterator
1)	The Iterator traverses the elements in the forward direction only.	ListIterator traverses the elements in backward and forward directions both.

NEWGEN CORPORATE TRAINING CENTER

2)	The Iterator can be used in List, Set, and Queue.	ListIterator can be used in List only.
3)	The Iterator can only perform remove operation while traversing the collection.	ListIterator can perform ?add, ?remove, ? and ?set? operation while traversing the collection.

67. What is the difference between List and Set?

- The List and Set both extend the collection interface.
- The List can contain duplicate elements whereas Set includes unique items.
- The List is an ordered collection which maintains the insertion order whereas Set is an unordered collection which does not preserve the insertion order.
- The List interface contains a single legacy class which is Vector class whereas Set interface does not have any legacy class.
- The List interface can allow n number of null values whereas Set interface only allows a single null value.

68. What is the difference between HashSet and TreeSet?

- The HashSet and TreeSet, both classes, implement Set interface. The differences between the both are listed below.
- HashSet maintains no order whereas TreeSet maintains ascending order.
- HashSet implemented by hash table whereas TreeSet implemented by a Tree structure.
- HashSet performs faster than TreeSet.

NEWGEN CORPORATE TRAINING CENTER

- HashSet is backed by HashMap whereas TreeSet is backed by TreeMap.
-

69. What is the difference between Set and Map?

The differences between the Set and Map are given below.

- Set contains values only whereas Map contains key and values both.
 - Set contains unique values whereas Map can contain unique Keys with duplicate values.
 - Set holds a single number of null value whereas Map can include a single null key with n number of null values.
-

70. What is the difference between HashSet and HashMap?

The differences between the HashSet and HashMap are listed below.

- HashSet contains only values whereas HashMap includes the entry (key, value). HashSet can be iterated, but HashMap needs to convert into Set to be iterated.
 - HashSet implements Set interface whereas HashMap implements the Map interface
 - HashSet cannot have any duplicate value whereas HashMap can contain duplicate values with unique keys.
 - HashSet contains the only single number of null value whereas HashMap can hold a single null key with n number of null values.
 -
-

71 What is the difference between HashMap and TreeMap?

The differences between the HashMap and TreeMap are given below.

- HashMap maintains no order, but TreeMap maintains ascending order.

NEWGEN CORPORATE TRAINING CENTER

- HashMap is implemented by hash table whereas TreeMap is implemented by a Tree structure.
- HashMap can be sorted by Key or value whereas TreeMap can be sorted by Key.
- HashMap may contain a null key with multiple null values whereas TreeMap cannot hold a null key but can have multiple null values.

72. What is the difference between HashMap and Hashtable?

No.	HashMap	Hashtable
1)	HashMap is not synchronized.	Hashtable is synchronized.
2)	HashMap can contain one null key and multiple null values.	Hashtable cannot contain any null key or null value.
3)	HashMap is not ?thread-safe,? so it is useful for non-threaded applications.	Hashtable is thread-safe, and it can be shared between various threads.
4)	4) HashMap inherits the AbstractMap class	Hashtable inherits the Dictionary class.

73. What is the difference between Collection and Collections?

The differences between the Collection and Collections are given below.

- The Collection is an interface whereas Collections is a class.
- The Collection interface provides the standard functionality of data structure to List, Set, and Queue. However, Collections class is to sort and synchronize the collection elements.

NEWGEN CORPORATE TRAINING CENTER

- The Collection interface provides the methods that can be used for data structure whereas Collections class provides the static methods which can be used for various operation on a collection.

74. When to use Abstraction and Interface ?

When to use an abstract class

- An abstract class is a good choice if we are using the inheritance concept since it provides a common base class implementation to derived classes.
- An abstract class is also good if we want to declare non-public members. In an interface, all methods must be public.
- If we want to add new methods in the future, then an abstract class is a better choice. Because if we add new methods to an interface, then all of the classes that already implemented that interface will have to be changed to implement the new methods.
- If we want to create multiple versions of our component, create an abstract class. Abstract classes provide a simple and easy way to version our components. By updating the base class, all inheriting classes are automatically updated with the change. Interfaces, on the other hand, cannot be changed once created. If a new version of an interface is required, we must create a whole new interface.
- Abstract classes have the advantage of allowing better forward compatibility. Once clients use an interface, we cannot change it; if they use an abstract class, we can still add behavior without breaking the existing code.
- If we want to provide common, implemented functionality among all implementations of our component, use an abstract class. Abstract classes allow us to partially implement our class, whereas interfaces contain no implementation for any members.

NEWGEN CORPORATE TRAINING CENTER

When to use an interface

- If the functionality we are creating will be useful across a wide range of disparate objects, use an interface. Abstract classes should be used primarily for objects that are closely related, whereas interfaces are best suited for providing a common functionality to unrelated classes.
- Interfaces are a good choice when we think that the API will not change for a while.
- Interfaces are also good when we want to have something similar to multiple inheritances since we can implement multiple interfaces.
- If we are designing small, concise bits of functionality, use interfaces. If we are designing large functional units, use an abstract class.

NEWGEN CORPORATE TRAINING CENTER